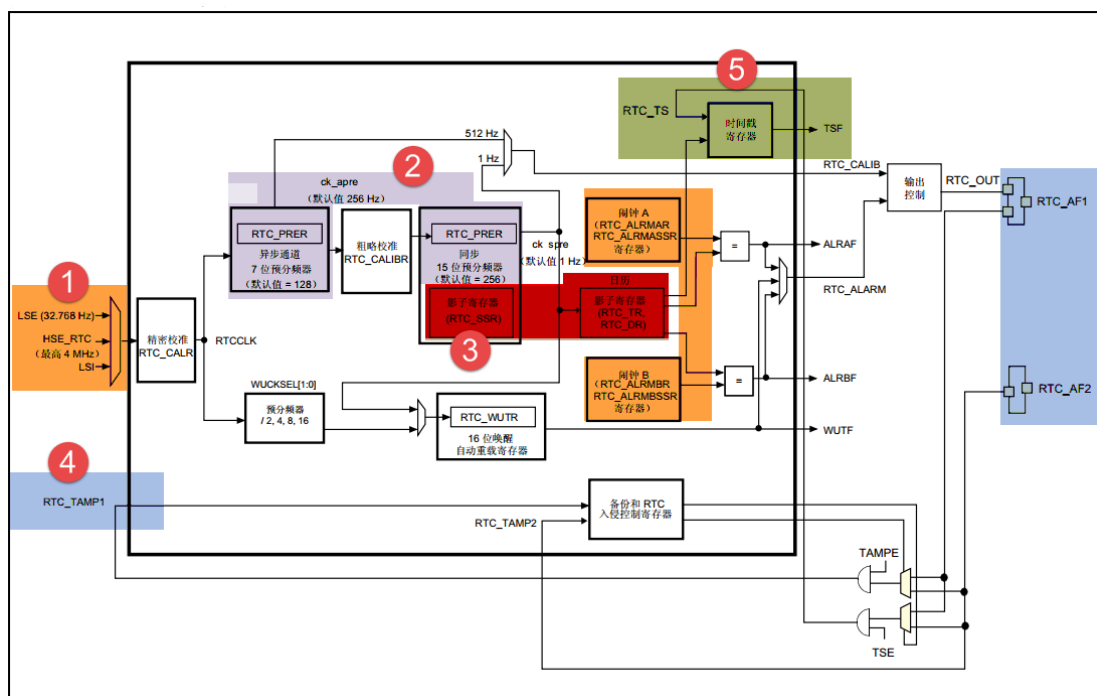


## 第47章 RTC—实时时钟

### 47.1 RTC 简介

RTC—real time clock，实时时钟，主要包含日历、闹钟和自动唤醒这三部分的功能，其中的日历功能我们使用的最多。日历包含两个 32bit 的时间寄存器，可直接输出时分秒，星期、月、日、年。比起 F103 系列的 RTC 只能输出秒中断，剩下的其他时间需要软件来实现，F407 的 RTC 可谓是脱胎换骨，让我们在软件编程时大大降低了难度。RTC 功能框图分析

### 47.2 RTC 功能框图解析



#### 1. 时钟源

RTC 时钟源—RTCCLK 可以从 LSE、LSI 和 HSE\_RTC 这三者中得到。其中使用最多的是 LSE，LSE 由一个外部的 32.768KHZ (6PF 负载) 的晶振提供，精度高，稳定，RTC 首选。LSI 是芯片内部的 30KHZ 晶体，精度较低，会有温漂，一般不建议使用。HSE\_RTC 由 HSE 分频得到，最高是 4M，使用的也较少。

#### 2. 预分频器

预分频器 PRER 由 7 位的异步预分频器 APRE 和 15 位的同步预分频器 SPRE 组成。异步预分频器时钟 CK\_APRE 用于为二进制 RTC\_SSRR 亚秒递减计数器提供时钟，同步预分频器时钟 CK\_SPRE 用于更新日历。异步预分频器时钟  $f_{CK\_APRE} = f_{RTC\_CLK} / (PREDIV\_A + 1)$ ，同步预分频器时钟  $f_{CK\_SPRE} = f_{RTC\_CLK} / (PREDIV\_S + 1)$ 。使用两个预分频器时，推荐将异步

预分频器配置为较高的值，以最大程度降低功耗。一般我们会使用 LSE 生成 1HZ 的同步预分频器时钟。通常的情况下，我们会选择 LSE 作为 RTC 的时钟源，即  $f_{\text{RTCCLK}}=f_{\text{LSE}}=32.768\text{KHZ}$ 。然后经过预分频器 PRER 分频生成 1HZ 的时钟用于更新日历。使用两个预分频器分频的时候，为了最大程度的降低功耗，我们一般把同步预分频器设置成较大的值，为了生成 1HZ 的同步预分频器时钟 CK\_SPRE，最常用的配置是  $\text{PREDIV\_A}=127$ ， $\text{PREDIV\_S}=255$ 。计算公式为： $f_{\text{CK\_SPRE}}=f_{\text{RTCCLK}}/\{(\text{PREDIV\_A}+1)*(\text{PREDIV\_S}+1)\}=32.768/\{(127+1)*(255+1)\}=1\text{HZ}$ 。

### 3. 实时时钟和日历

我们知道，实时时钟一般是这样表示的：时/分/秒/亚秒，其中时分秒可直接从 RTC 时间寄存器 (RTC\_TR) 中读取，有关时间寄存器的说明具体见图 47-1 和表格 47-1。

|          |          |    |    |          |    |    |    |          |         |         |    |         |    |    |    |
|----------|----------|----|----|----------|----|----|----|----------|---------|---------|----|---------|----|----|----|
| 31       | 30       | 29 | 28 | 27       | 26 | 25 | 24 | 23       | 22      | 21      | 20 | 19      | 18 | 17 | 16 |
| Reserved |          |    |    |          |    |    |    |          | PM      | HT[1:0] |    | HU[3:0] |    |    |    |
|          |          |    |    |          |    |    |    |          | rw      | rw      | rw | rw      | rw | rw | rw |
| 15       | 14       | 13 | 12 | 11       | 10 | 9  | 8  | 7        | 6       | 5       | 4  | 3       | 2  | 1  | 0  |
| Reserved | MNT[2:0] |    |    | MNU[3:0] |    |    |    | Reserved | ST[2:0] |         |    | SU[3:0] |    |    |    |
|          | rw       | rw | rw | rw       | rw | rw | rw |          | rw      | rw      | rw | rw      | rw | rw | rw |

图 47-1 RTC 时间寄存器 (RTC\_TR)

表格 47-1 时间寄存器位功能说明

| 位名称      | 位说明                       |
|----------|---------------------------|
| PM       | AM/PM 符号，0:AM/24 小时制，1:PM |
| HT[1:0]  | 小时的十位                     |
| HU[3:0]  | 小时的个位                     |
| MNT[2:0] | 分钟的十位                     |
| MNU[3:0] | 分钟的个位                     |
| ST[2:0]  | 秒的十位                      |
| SU[3:0]  | 秒的个位                      |

亚秒由 RTC 亚秒寄存器 (RTC\_SSR) 的值计算得到，公式为：亚秒值 =  $(\text{PREDIV\_S} - \text{SS}[15:0]) / (\text{PREDIV\_S} + 1)$ ，SS[15:0] 是同步预分频器计数器的值，PREDIV\_S 是同步预分频器的值。

|          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| r        | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| SS[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| r        | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  |

图 47-2 RTC 亚秒寄存器 (RTC\_SSR)

日期包含的年月日可直接从 RTC 日期寄存器 (RTC\_DR) 中读取。

|          |    |    |    |    |         |    |    |         |          |    |         |         |         |    |    |
|----------|----|----|----|----|---------|----|----|---------|----------|----|---------|---------|---------|----|----|
| 31       | 30 | 29 | 28 | 27 | 26      | 25 | 24 | 23      | 22       | 21 | 20      | 19      | 18      | 17 | 16 |
| Reserved |    |    |    |    |         |    |    | YT[3:0] |          |    |         | YU[3:0] |         |    |    |
|          |    |    |    |    |         |    |    | rw      | rw       | rw | rw      | rw      | rw      | rw | rw |
| 15       | 14 | 13 | 12 | 11 | 10      | 9  | 8  | 7       | 6        | 5  | 4       | 3       | 2       | 1  | 0  |
| WDU[2:0] |    |    |    | MT | MU[3:0] |    |    |         | Reserved |    | DT[1:0] |         | DU[3:0] |    |    |
| rw       | rw | rw | rw | rw | rw      | rw | rw | rw      |          |    | rw      | rw      | rw      | rw | rw |

图 47-3 RTC 日期寄存器 (RTC\_DR)

表格 47-2 RTC 日期寄存器位功能说明

| 位名称      | 位说明                               |
|----------|-----------------------------------|
| YT[1:0]  | 年份的十位                             |
| YU[3:0]  | 年份的个位                             |
| WDU[2:0] | 星期几的个位，000：禁止，001：星期一，...，111：星期日 |
| MT       | 月份的十位                             |
| MU       | 月份的个位                             |
| DT[1:0]  | 日期的十位                             |
| DU[3:0]  | 日期的个位                             |

当应用程序读取日历寄存器时，默认是读取影子寄存器的内容，每隔两个 RTCCLK 周期，便将当前日历值复制到影子寄存器。我们也可以通过将 RTC\_CR 寄存器的 BYPSHAD 控制位置 1 来直接访问日历寄存器，这样可避免等待同步的持续时间。

RTC\_CLK 经过预分频器后，有一个 512HZ 的 CK\_APRE 和 1 个 1HZ 的 CK\_SPRE，这两个时钟可以成为校准的时钟输出 RTC\_CALIB，RTC\_CALIB 最终要输出则需映射到 RTC\_AF1 引脚，即 PC13 输出，用来对外部提供时钟。

## 4. 闹钟

RTC 有两个闹钟，闹钟 A 和闹钟 B，当 RTC 运行的时间跟预设的闹钟时间相同的时候，相应的标志位 ALRAF（在 RTC\_ISR 寄存器中）和 ALRBF 会置 1。利用这个闹钟我们可以做一些备忘提醒功能。

如果使能了闹钟输出（由 RTC\_CR 的 OSEL[0:1]位控制），则 ALRAF 和 ALRBF 会连接到闹钟输出引脚 RTC\_ALARM，RTC\_ALARM 最终连接到 RTC 的外部引脚 RTC\_AF1（即 PC13），输出的极性由 RTC\_CR 寄存器的 POL 位配置，可以是高电平或者低电平。

## 5. 时间戳

时间戳即时间点的意思，就是某一个时刻的时间。时间戳复用功能 (RTC\_TS) 可映射到 RTC\_AF1 或 RTC\_AF2，当发生外部的入侵事件时，即发生时间戳事件时，RTC\_ISR 寄存器中的时间戳标志位 (TSF) 将置 1，日历会保存到时间戳寄存器（RTC\_TSSSR、RTC\_TSTR 和 RTC\_TSDR）中。时间戳往往用来记录危及时刻的时间，以供事后排查问题时查询。

## 6. 入侵检测

RTC 自带两个入侵检测引脚 RTC\_AF1（PC13）和 RTC\_AF2（PI8，PI8 只有在 176pin 引脚的型号中才有），这两个输入既可配置为边沿检测，也可配置为带过滤的电平检测。当发

生入侵检测时，备份寄存器将被复位。备份寄存器 (RTC\_BKPxR) 包括 20 个 32 位寄存器，用于存储 80 字节的用户应用数据。这些寄存器在备份域中实现，可在 VDD 电源关闭时通过 VBAT 保持上电状态。备份寄存器不会在系统复位或电源复位时复位，也不会当器件从待机模式唤醒时复位。

## 47.3 RTC 初始化结构体讲解

标准库函数对每个外设都建立了一个初始化结构体，比如 RTC\_InitTypeDef，结构体成员用于设置外设工作参数，并由外设初始化配置函数，比如 RTC\_Init()调用，这些配置好的参数将会设置外设相应的寄存器，达到配置外设工作环境的目的。

初始化结构体和初始化库函数配合使用是标准库精髓所在，理解了初始化结构体每个成员意义基本上就可以对该外设运用自如。初始化结构体定义在 stm32f4xx\_rtc.h 头文件中，初始化库函数定义在 stm32f4xx\_rtc.c 文件中，编程时我们可以结合这两个文件内注释使用。

RTC 初始化结构体用来设置 RTC 小时的格式和 RTC\_CLK 的分频系数。

代码清单 47-1 RTC 初始化结构体

```
1 typedef struct {  
2     uint32_t RTC_HourFormat;    /* 指定 RTC 小时格式*/  
3  
4     uint32_t RTC_AsynchPrediv; /* 配置 RTC_CLK 的异步分频因子 */  
5  
6     uint32_t RTC_SynchPrediv;  /* 配置 RTC_CLK 的同步分频因子 */  
7 } RTC_InitTypeDef;
```

- 1) RTC\_HourFormat：小时格式设置，有 RTC\_HourFormat\_24 和 RTC\_HourFormat\_12 两种格式，一般我们选择使用 24 小时制，具体由 RTC\_CR 寄存器的 FMT 位配置。
- 2) RTC\_AsynchPrediv: RTC\_CLK 异步分频因子设置，7 位有效，具体由 RTC 预分频器寄存器 RTC\_PRER 的 PREDIV\_A[6:0]位配置。
- 3) RTC\_SynchPrediv: RTC\_CLK 同步分频因子设置，15 位有效，具体由 RTC 预分频器寄存器 RTC\_PRER 的 PREDIV\_S[14:0]位配置。

## 47.4 RTC 时间结构体讲解

RTC 时间结构体用来设置初始时间，配置的是 RTC 时间寄存器 RTC\_TR。

代码 47-1 RTC 时间结构体

```
1 typedef struct {  
2     uint8_t RTC_Hours;    /* 小时设置 */  
3  
4     uint8_t RTC_Minutes;  /* 分钟设置 */  
5  
6     uint8_t RTC_Seconds;  /* 秒设置 */  
7  
8     uint8_t RTC_H12;      /* AM/PM 符号设置 */  
9 } RTC_TimeTypeDef;
```

- 1) RTC\_Hours: 小时设置, 12 小时制式时, 取值范围为 0~11, 24 小时制式时, 取值范围为 0~23。
- 2) RTC\_Minutes: 分钟设置, 取值范围为 0~59。
- 3) RTC\_Seconds: 秒钟设置, 取值范围为 0~59。
- 4) RTC\_H12: AM/PM 设置, 可取值 RTC\_H12\_AM 和 RTC\_H12\_PM, RTC\_H12\_AM 时则是 24 小时制, RTC\_H12\_PM 则是 12 小时制。

## 47.5 RTC 日期结构体讲解

RTC 日期结构体用来设置初始日期, 配置的是 RTC 日期寄存器 RTC\_DR。

代码 47-2 RTC 日期结构体

```
1 typedef struct {  
2     uint8_t RTC_WeekDay; /* 星期几设置 */  
3  
4     uint8_t RTC_Month; /* 月份设置 */  
5  
6     uint8_t RTC_Date; /* 日期设置 */  
7  
8     uint8_t RTC_Year; /* 年份设置 */  
9 } RTC_DateTypeDef;
```

- 1) RTC\_WeekDay: 星期几设置, 取值范围为 1~7, 对应星期一~星期日。
- 2) RTC\_Month: 月份设置, 取值范围为 1~12。
- 3) RTC\_Date: 日期设置, 取值范围为 1~31。
- 4) RTC\_Year: 年份设置, 取值范围为 0~99。

## 47.6 RTC 闹钟结构体讲解

RTC 闹钟结构体主要用来设置闹钟时间, 设置的格式为[星期/日期]-[时]-[分]-[秒], 共四个字段, 每个字段都可以设置为有效或者无效, 即可 MASK。如果 MASK 掉[星期/日期]字段, 则每天闹钟都会响。

代码 47-3 RTC 闹钟结构体

```
1 typedef struct {  
2     RTC_TimeTypeDef RTC_AlarmTime; /* 设定 RTC 时间寄存器的值: 时/分/秒 */  
3  
4     uint32_t RTC_AlarmMask; /* RTC 闹钟 掩码字段选择 */  
5  
6     uint32_t RTC_AlarmDateWeekDaySel; /* 闹钟的日期/星期选择 */  
7  
8     uint8_t RTC_AlarmDateWeekDay; /* 指定闹钟的日期/星期  
9                                     * 如果日期有效, 则取值范围为 1~31  
10                                    * 如果星期有效, 则取值为 1~7  
11                                    */  
12 } RTC_AlarmTypeDef;
```

- 1) RTC\_AlarmTime: 闹钟时间设置, 配置的是 RTC 时间初始化结构体, 主要配置小时的制式, 有 12 小时或者是 24 小时, 配套具体的时、分、秒。
- 2) RTC\_AlarmMask: 闹钟掩码字段选择, 即选择闹钟时间哪些字段无效, 取值 可为: RTC\_AlarmMask\_None(全部有效)、RTC\_AlarmMask\_DateWeekDay (日期或者星期无效)、RTC\_AlarmMask\_Hours (小时无效)、RTC\_AlarmMask\_Minutes (分钟无效)、RTC\_AlarmMask\_Seconds (秒钟无效)、RTC\_AlarmMask\_All (全部无效)。比如我们选择 RTC\_AlarmMask\_DateWeekDay, 那么就是当 RTC 的时间的小时等于闹钟时间小时字段时, 每天的这个小时都会产生闹钟中断。
- 3) RTC\_AlarmDateWeekDaySel : 闹钟日期或者星期选择, 可选择 RTC\_AlarmDateWeekDaySel\_WeekDay 或者 RTC\_AlarmDateWeekDaySel\_Date。要想这个配置有效, 则 RTC\_AlarmMask 不能配置为 RTC\_AlarmMask\_DateWeekDay, 否则会被 MASK 掉。
- 4) RTC\_AlarmDateWeekDay: 具体的日期或者星期几, 当 RTC\_AlarmDateWeekDaySel 设置成 RTC\_AlarmDateWeekDaySel\_WeekDay 时, 取值为 1~7, 对应星期一~星期日, 当设置成 RTC\_AlarmMask\_DateWeekDay 时, 取值为 1~31。

## 47.7 RTC—日历实验

利用 RTC 的日历功能制作一个日历, 显示格式为: 年-月-日-星期, 时-分-秒。

### 47.7.1 硬件设计

该实验用到了片内外设 RTC, 为了确保在 VDD 断电的情况下时间可以保存且继续运行, VBAT 引脚外接了一个 CR1220 电池座, 用来放 CR1220 电池给 RTC 供电, 电路图具体见图 47-4。

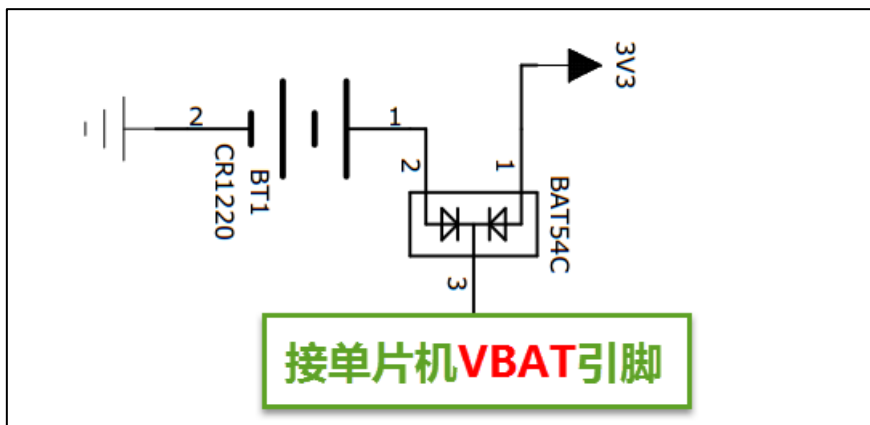


图 47-4 RTC 外接 CR1220 电池座子

## 47.7.2 软件设计

### 1. 编程要点

- 1) 选择 RTC\_CLK 的时钟源;
- 2) 配置 RTC\_CLK 的分频系数, 包括异步和同步两个;
- 3) 设置初始时间, 包括日期;
- 4) 获取时间和日期, 并显示;

### 2. 代码分析

这里只讲解核心的部分代码, 有些变量的设置, 头文件的包含等并没有涉及到, 完整的代码请参考本章配套的工程。我们创建了两个文件: bsp\_rtc.c 和 bsp\_rtc.h 文件用来存 RTC 驱动程序及相关宏定义, 中断服务函数则放在 stm32f4xx\_it.h 文件中。

#### 宏定义

##### 代码清单 47-2 宏定义

```
1 //是否使用 LCD 显示
2 // #define USE_LCD_DISPLAY
3
4 // 时钟源宏定义
5 // #define RTC_CLOCK_SOURCE_LSE
6 #define RTC_CLOCK_SOURCE_LSI
7
8 // 异步分频因子
9 #define ASYNCHPREDIV          0X7F
10 // 同步分频因子
11 #define SYNCHPREDIV           0XFF
12
13 // 时间宏定义
14 #define RTC_H12_AMorPM       RTC_H12_AM
15 #define HOURS                 1          // 0~23
16 #define MINUTES               1          // 0~59
17 #define SECONDS               1          // 0~59
18
19 // 日期宏定义
20 #define WEEKDAY               1          // 1~7
21 #define DATE                  1          // 1~31
22 #define MONTH                 1          // 1~12
23 #define YEAR                  1          // 0~99
24
25 // 时间格式宏定义
26 #define RTC_Format_BINorBCD   RTC_Format_BIN
27
28 // 备份域寄存器宏定义
29 #define RTC_BKP_DRX           RTC_BKP_DR0
30 // 写入到备份寄存器的数据宏定义
31 #define RTC_BKP_DATA          0X32F2
```



为了方便程序移植，我们把移植时需要修改的代码部分都通过宏定义来实现。具体的配合注释看代码即可。

## RTC 时钟配置函数

### 代码清单 47-3 RTC 时钟配置函数

```
1  /**
2   * @brief  RTC 配置：选择 RTC 时钟源，设置 RTC_CLK 的分频系数
3   * @param  无
4   * @retval 无
5   */
6  void RTC_CLK_Config(void)
7  {
8      RTC_InitTypeDef RTC_InitStructure;
9
10     /*使能 PWR 时钟*/
11     RCC_APB1PeriphClockCmd(RCC_APB1Periph_PWR, ENABLE);
12     /* PWR_CR:DBF 置 1，使能 RTC、RTC 备份寄存器和备份 SRAM 的访问 */
13     PWR_BackupAccessCmd(ENABLE);
14
15     #if defined (RTC_CLOCK_SOURCE_LSI)
16         /* 使用 LSI 作为 RTC 时钟源会有误差,仅仅是为了实验方便 */
17
18         /* 使能 LSI */
19         RCC_LSICmd(ENABLE);
20         /* 等待 LSI 稳定 */
21         while (RCC_GetFlagStatus(RCC_FLAG_LSIRDY) == RESET) {
22         }
23         /* 选择 LSI 做为 RTC 的时钟源 */
24         RCC_RTCCLKConfig(RCC_RTCCLKSource_LSI);
25
26     #elif defined (RTC_CLOCK_SOURCE_LSE)
27         /* 使能 LSE */
28         RCC_LSEConfig(RCC_LSE_ON);
29         /* 等待 LSE 稳定 */
30         while (RCC_GetFlagStatus(RCC_FLAG_LSERDY) == RESET) {
31         }
32         /* 选择 LSE 做为 RTC 的时钟源 */
33         RCC_RTCCLKConfig(RCC_RTCCLKSource_LSE);
34
35     #endif /* RTC_CLOCK_SOURCE_LSI */
36
37     /* 使能 RTC 时钟 */
38     RCC_RTCCLKCmd(ENABLE);
39
40     /* 等待 RTC APB 寄存器同步 */
41     RTC_WaitForSynchro();
42
43     /*=====初始化同步/异步预分频器的值=====*/
44     /* 驱动日历的时钟 ck_spare = LSE/[(255+1)*(127+1)] = 1HZ */
45
46     /* 设置异步预分频器的值*/
47     RTC_InitStructure.RTC_AsynchPrediv = ASYNCHPREDIV;
48     /* 设置同步预分频器的值 */
49     RTC_InitStructure.RTC_SynchPrediv = SYNCHPREDIV;
50     RTC_InitStructure.RTC_HourFormat = RTC_HourFormat_24;
51     /* 用 RTC_InitStructure 的内容初始化 RTC 寄存器 */
52     if (RTC_Init(&RTC_InitStructure) == ERROR) {
53         printf("\n\r RTC 时钟初始化失败 \r\n");
54     }
55 }
56
57 }
```



RTC 时钟配置函数主要实现两个功能，一是选择 RTC\_CLK 的时钟源，根据宏定义来决定使用 LSE 还是 LSI 作为 RTC\_CLK 的时钟源，这里为了方便我们选择 LSI；二是设置 RTC\_CLK 的预分频系数，包括异步和同步两个，这里设置异步分频因子为 ASYNCHPREDIV（127），同步分频因子为 ASYNCHPREDIV（255），则产生的最终驱动日历的时钟  $CK\_SPRE=32.768/(127+1)*(255+1)=1\text{HZ}$ ，则每秒更新一次。

### RTC 时间初始化函数

代码清单 47-4 RTC 时间和日期设置函数

```
1 /**
2  * @brief  设置时间和日期
3  * @param  无
4  * @retval 无
5  */
6 void RTC_TimeAndDate_Set(void)
7 {
8     RTC_TimeTypeDef RTC_TimeStructure;
9     RTC_DateTypeDef RTC_DateStructure;
10
11     // 初始化时间
12     RTC_TimeStructure.RTC_H12 = RTC_H12_AMorPM;
13     RTC_TimeStructure.RTC_Hours = HOURS;
14     RTC_TimeStructure.RTC_Minutes = MINUTES;
15     RTC_TimeStructure.RTC_Seconds = SECONDS;
16     RTC_SetTime(RTC_Format_BINorBCD, &RTC_TimeStructure);
17     RTC_WriteBackupRegister(RTC_BKP_DRX, RTC_BKP_DATA);
18
19     // 初始化日期
20     RTC_DateStructure.RTC_WeekDay = WEEKDAY;
21     RTC_DateStructure.RTC_Date = DATE;
22     RTC_DateStructure.RTC_Month = MONTH;
23     RTC_DateStructure.RTC_Year = YEAR;
24     RTC_SetDate(RTC_Format_BINorBCD, &RTC_DateStructure);
25     RTC_WriteBackupRegister(RTC_BKP_DRX, RTC_BKP_DATA);
26 }
```

RTC 时间和日期设置函数主要是设置时间和日期这两个结构体，然后调相应的 RTC\_SetTime 和 RTC\_SetDate 函数把初始化好的时间写到相应的寄存器，每当写完之后都会在备份寄存器里面写入一个数，以作标记，为的是程序开始运行的时候检测 RTC 的时间是否已经配置过。

具体的时间、日期、备份寄存器和写入备份寄存器的值都在头文件的宏定义里面，要修改这些值只需修改头文件的宏定义即可。

### RTC 时间显示函数

代码清单 47-5 RTC 时间显示函数

```
1 /**
2  * @brief  显示时间和日期
3  * @param  无
4  * @retval 无
5  */
6 void RTC_TimeAndDate_Show(void)
7 {
8     uint8_t RtcTmp=0;
9     char LCDTemp[100];
10     RTC_TimeTypeDef RTC_TimeStructure;
11     RTC_DateTypeDef RTC_DateStructure;
12
13 }
```

```
14     while (1) {
15         // 获取日历
16         RTC_GetTime(RTC_Format_BIN, &RTC_TimeStructure);
17         RTC_GetDate(RTC_Format_BIN, &RTC_DateStructure);
18
19         // 每秒打印一次
20         if (RtcTmp != RTC_TimeStructure.RTC_Seconds) {
21
22             // 打印日期
23             printf("The Date : Y:20%0.2d - M:%0.2d - D:%0.2d - W:%0.2d\r\n",
24                 RTC_DateStructure.RTC_Year,
25                 RTC_DateStructure.RTC_Month,
26                 RTC_DateStructure.RTC_Date,
27                 RTC_DateStructure.RTC_WeekDay);
28
29             // 液晶显示日期
30             // 先把要显示的数据用 sprintf 函数转换为字符串, 然后才能用液晶显示函数显示
31             sprintf(LCDTemp, "The Date : Y:20%0.2d - M:%0.2d - D:%0.2d - W:%0.2d",
32                 RTC_DateStructure.RTC_Year,
33                 RTC_DateStructure.RTC_Month,
34                 RTC_DateStructure.RTC_Date,
35                 RTC_DateStructure.RTC_WeekDay);
36
37             #ifdef USE_LCD_DISPLAY
38                 LCD_DisplayStringLineEx(10, 50, 48, 48, (uint8_t *)LCDTemp, 0);
39             #endif
40
41             // 打印时间
42             printf("The Time : %0.2d:%0.2d:%0.2d \r\n\r\n",
43                 RTC_TimeStructure.RTC_Hours,
44                 RTC_TimeStructure.RTC_Minutes,
45                 RTC_TimeStructure.RTC_Seconds);
46
47             // 液晶显示时间
48             sprintf(LCDTemp, "The Time : %0.2d:%0.2d:%0.2d",
49                 RTC_TimeStructure.RTC_Hours,
50                 RTC_TimeStructure.RTC_Minutes,
51                 RTC_TimeStructure.RTC_Seconds);
52
53             #ifdef USE_LCD_DISPLAY
54                 LCD_DisplayStringLineEx(10, 100, 48, 48, (uint8_t *)LCDTemp, 0);
55             #endif
56             (void)RTC->DR;
57         }
58         RtcTmp = RTC_TimeStructure.RTC_Seconds;
59     }
60 }
```

RTC 时间和日期显示函数中, 通过调用 `RTC_GetTime()`和 `RTC_GetDate()`这两个库函数, 把时间和日期都读取保存到时间和日期结构体中, 然后以 1s 为频率, 把时间显示出来。

在使用液晶显示时间的时候, 需要先调用标准的 C 库函数 `sprintf()`把数据转换成字符串, 然后才能调用液晶显示函数来显示, 因为液晶显示时处理的都是字符串。

## 主函数

### 代码清单 47-6 main 函数

```
1 #include "stm32f4xx.h"
2 #include "../led/bsp_led.h"
3 #include "../usart/bsp_debug_usart.h"
4 #include "../RTC/bsp_rtc.h"
5
6 /**
7  * @brief 主函数
8  * @param 无
9  * @retval 无
```

```
10  */
11 int main(void)
12 {
13     /* 初始化 LED */
14     LED_GPIO_Config();
15
16     /* 初始化调试串口，一般为串口 1 */
17     Debug_USART_Config();
18     printf("\n\r 这是一个 RTC 日历实验 \r\n");
19
20 #ifdef USE_LCD_DISPLAY
21     /*=====液晶初始化开始=====*/
22     LCD_Init();
23     LCD_LayerInit();
24     LTDC_Cmd(ENABLE);
25
26     /*把背景层刷黑色*/
27     LCD_SetLayer(LCD_BACKGROUND_LAYER);
28     LCD_Clear(LCD_COLOR_BLACK);
29
30     /*初始化后默认使用前景层*/
31     LCD_SetLayer(LCD_FOREGROUND_LAYER);
32     /*默认设置不透明，该函数参数为不透明度，范围 0-0xff，0 为全透明，0xff 为不透明*/
33     LCD_SetTransparency(0xFF);
34     LCD_Clear(LCD_COLOR_BLACK);
35     /*经过 LCD_SetLayer(LCD_FOREGROUND_LAYER) 函数后，
36     以下液晶操作都在前景层刷新，除非重新调用过 LCD_SetLayer 函数设置背景层*/
37     LCD_SetColors(LCD_COLOR_RED, LCD_COLOR_BLACK);
38     /*=====液晶初始化结束=====*/
39 #endif
40     /*
41     * 当我们配置过 RTC 时间之后就往备份寄存器 0 写入一个数据做标记
42     * 所以每次程序重新运行的时候就通过检测备份寄存器 0 的值来判断
43     * RTC 是否已经配置过，如果配置过那就继续运行，如果没有配置过
44     * 就初始化 RTC，配置 RTC 的时间。
45     */
46
47     /* RTC 配置：选择时钟源，设置 RTC_CLK 的分频系数 */
48     RTC_CLK_Config();
49
50     if (RTC_ReadBackupRegister(RTC_BKP_DRX) != RTC_BKP_DATA) {
51         /* 设置时间和日期 */
52         RTC_TimeAndDate_Set();
53     } else {
54         /* 检查是否电源复位 */
55         if (RCC_GetFlagStatus(RCC_FLAG_PORRST) != RESET) {
56             printf("\r\n 发生电源复位....\r\n");
57         }
58         /* 检查是否外部复位 */
59         else if (RCC_GetFlagStatus(RCC_FLAG_PINRST) != RESET) {
60             printf("\r\n 发生外部复位....\r\n");
61         }
62
63         printf("\r\n 不需要重新配置 RTC....\r\n");
64
65         /* 使能 PWR 时钟 */
66         RCC_APB1PeriphClockCmd(RCC_APB1Periph_PWR, ENABLE);
67         /* PWR_CR:DBF 置 1，使能 RTC、RTC 备份寄存器和备份 SRAM 的访问 */
68         PWR_BackupAccessCmd(ENABLE);
69         /* 等待 RTC APB 寄存器同步 */
70         RTC_WaitForSynchro();
71     }
72 }
```

```
73  /* 显示时间和日期 */
74  RTC_TimeAndDate_Show();
75 }
```

主函数中，我们调用 `RTC_ReadBackupRegister()` 库函数来读取备份寄存器的值是否等于我们预设的那个值，因为当我们初始化完 RTC 的时间之后就往备份寄存器写入一个数据做标记，所以每次程序重新运行的时候就通过检测备份寄存器的值来判断，RTC 是否已经配置过，如果配置过则判断是电源复位还是外部引脚复位且继续运行显示时间，如果没有配置过，就初始化 RTC，配置 RTC 的时间，然后显示。

如果想每次程序运行时都重新配置 RTC，则用一个异于写入的值来做判断即可。

### 3. 下载验证

把程序编译好下载到开发板，通过电脑端口的串口调试助手或者液晶可以看到时间正常运行。当 VDD 不断电的情况下，发生外部引脚复位，时间不会丢失。当 VDD 断电或者发生外部引脚复位，VBT 有电池供电时，时间不会丢失。当 VDD 断电且 VBAT 也不供电的情况下，时间会丢失，然后根据程序预设的初始时间重新启动。

## 47.8 RTC—闹钟实验

在日历实验的基础上，利用 RTC 的闹钟功能制作一个闹钟，在每天的[XX 小时-XX 分钟-XX 秒钟]产生闹钟，然后蜂鸣器响。

### 47.8.1 硬件设计

硬件设计跟日历实验部分的硬件设计一样。

### 47.8.2 软件设计

闹钟实验是在日历实验的基础上添加，相同部分的代码不再讲解，这里只讲解闹钟相关的代码，更加具体的请参考闹钟实验的工程源码。

#### 闹钟相关宏定义

##### 代码清单 47-7 闹钟相关宏定义

```
1 // 闹钟相关宏定义
2 #define ALARM_HOURS          1           // 0~23
3 #define ALARM_MINUTES       1           // 0~59
4 #define ALARM_SECONDS       10          // 0~59
5
6 #define ALARM_MASK            RTC_AlarmMask_DateWeekDay
7 #define ALARM_DATE_WEEKDAY_SEL  RTC_AlarmDateWeekDaySel_WeekDay
8 #define ALARM_DATE_WEEKDAY    2
9 #define RTC_Alarm_X            RTC_Alarm_A
```

为了方便程序移植，我们把需要频繁修改的代码用宏封装起来。如果需要设置闹钟时间和闹钟的掩码字段，只需要修改这些宏即可。这些宏对应的是 RTC 闹钟结构体的成员，想知道每个宏的具体含义可参考“RTC 闹钟结构体讲解”小节。

闹钟时间字段掩码 `ALARM_MASK` 我们配置为 `MASK` 掉日期/星期，即忽略日期/星期，则闹钟时间只有时/分/秒有效，即每天到了这个时间闹钟都会响。掩码还有其他取值，用户可自行修改做实验。

## 1. 编程要点

- 1) 初始化 RTC，设置 RTC 初始时间；
- 2) 编程闹钟，设置闹钟时间；
- 3) 编写闹钟中断服务函数；

## 2. 代码分析

### 闹钟设置函数

#### 代码 47-4 闹钟编程代码

```
1 /*
2  *  要使能 RTC 闹钟中断，需按照以下顺序操作：
3  *  1. 将 EXTI 线 17 配置为中断模式并将其使能，然后选择上升沿有效。
4  *  2. 配置 NVIC 中的 RTC_Alarm_IRQ 通道并将其使能。
5  *  3. 配置 RTC 以生成 RTC 闹钟（闹钟 A 或闹钟 B）。
6  *
7  *
8  */
9 void RTC_AlarmSet(void)
10 {
11     NVIC_InitTypeDef  NVIC_InitStructure;
12     EXTI_InitTypeDef  EXTI_InitStructure;
13     RTC_AlarmTypeDef  RTC_AlarmStructure;
14
15     /*=====第①步=====*/
16     /* RTC 闹钟中断配置 */
17     /* EXTI 配置 */
18     EXTI_ClearITPendingBit(EXTI_Line17);
19     EXTI_InitStructure.EXTI_Line = EXTI_Line17;
20     EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
21     EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Rising;
22     EXTI_InitStructure.EXTI_LineCmd = ENABLE;
23     EXTI_Init(&EXTI_InitStructure);
24
25     /*=====第②步=====*/
26     /* 使能 RTC 闹钟中断 */
27     NVIC_InitStructure.NVIC_IRQChannel = RTC_Alarm_IRQn;
28     NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
29     NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0;
30     NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
31     NVIC_Init(&NVIC_InitStructure);
32
33     /*=====第③步=====*/
34     /* 失能闹钟，在设置闹钟时间的时候必须先失能闹钟 */
35     RTC_AlarmCmd(RTC_Alarm_X, DISABLE);
36     /* 设置闹钟时间 */
37     RTC_AlarmStructure.RTC_AlarmTime.RTC_H12 = RTC_H12_AMorPM;
38     RTC_AlarmStructure.RTC_AlarmTime.RTC_Hours = ALARM_HOURS;
39     RTC_AlarmStructure.RTC_AlarmTime.RTC_Minutes = ALARM_MINUTES;
40     RTC_AlarmStructure.RTC_AlarmTime.RTC_Seconds = ALARM_SECONDS;
41     RTC_AlarmStructure.RTC_AlarmMask = ALARM_MASK;
42     RTC_AlarmStructure.RTC_AlarmDateWeekDaySel = ALARM_DATE_WEEKDAY_SEL;
43     RTC_AlarmStructure.RTC_AlarmDateWeekDay = ALARM_DATE_WEEKDAY;
44
45
46     /* 配置 RTC Alarm X (X=A 或 B) 寄存器 */
```

```
47    RTC_SetAlarm(RTC_Format_BINorBCD, RTC_Alarm_X, &RTC_AlarmStructure);
48
49    /* 使能 RTC Alarm X 中断 */
50    RTC_ITConfig(RTC_IT_ALRA, ENABLE);
51
52    /* 使能闹钟 */
53    RTC_AlarmCmd(RTC_Alarm_X, ENABLE);
54
55    /* 清除闹钟中断标志位 */
56    RTC_ClearFlag(RTC_FLAG_ALRAF);
57    /* 清除 EXTI Line 17 悬起位 (内部连接到 RTC Alarm) */
58    EXTI_ClearITPendingBit(EXTI_Line17);
59 }
```

从参考手册知道，要使能 RTC 闹钟中断，必须按照三个步骤进行，具体见图 47-5。RTC\_AlarmSet()函数则根据这三个步骤和代码中的注释阅读即可。

## RTC 中断

所有 RTC 中断均与 EXTI 控制器相连。

要使能 RTC 闹钟中断，需按照以下顺序操作：

1. 将 EXTI 线 17 配置为中断模式并将其使能，然后选择上升沿有效。
2. 配置 NVIC 中的 RTC\_Alarm IRQ 通道并将其使能。
3. 配置 RTC 以生成 RTC 闹钟（闹钟 A 或闹钟 B）。

图 47-5 RTC 闹钟中断编程步骤（摘自 STM32F4xx 参考手册 RTC 章节）

在第 3 步中，配置 RTC 以生成 RTC 闹钟中，在手册中也有详细的步骤说明，编程的时候必须按照这个步骤，具体见图 47-6。

### 编程闹钟

要对可编程的闹钟（闹钟 A 或闹钟 B）进行编程或更新，必须执行类似的步骤：

1. 将 RTC\_CR 寄存器中的 ALRAE 或 ALRBE 位清零以禁止闹钟 A 或闹钟 B。
2. 轮询 RTC\_ISR 寄存器中的 ALRAWF 或 ALRBWF 位，直到其中一个置 1，以确保闹钟寄存器可以访问。大约需要 2 个 RTCCLK 时钟周期（由于时钟同步）。
3. 编程闹钟 A 或闹钟 B 寄存器（RTC\_ALRMASSR/RTC\_ALRMAR 或 RTC\_ALRMBSSR/RTC\_ALRMBR）。
4. 将 RTC\_CR 寄存器中的 ALRAE 或 ALRBE 位置 1 以再次使能闹钟 A 或闹钟 B。

图 47-6 RTC 闹钟编程步骤（摘自 STM32F4xx 参考手册 RTC 章节）

编程闹钟的步骤 1 和 2，由固件库函数 RTC\_AlarmCmd(RTC\_Alarm\_X, DISABLE);实现，即在编程闹钟寄存器设置闹钟时间的时候必须先失能闹钟。剩下的两个步骤配套代码的注释阅读即可。

### 闹钟中断服务函数

代码清单 47-8 闹钟中断服务函数

```
1 // 闹钟中断服务函数
2 void RTC_Alarm_IRQHandler(void)
3 {
4     if (RTC_GetITStatus(RTC_IT_ALRA) != RESET) {
5         RTC_ClearITPendingBit(RTC_IT_ALRA);
6         EXTI_ClearITPendingBit(EXTI_Line17);
7     }
```

```
8      /* 闹钟时间到，蜂鸣器响 */
9      BEEP_ON;
10 }
```

如果日历时间到了闹钟设置好的时间，则产生闹钟中断，在中断函数中把相应的标志位清 0。为了表示闹钟时间到，我们让蜂鸣器响。

## main 函数

### 代码清单 47-9 main 函数

```
1 #include "stm32f4xx.h"
2 #include "../led/bsp_led.h"
3 #include "../usart/bsp_debug_usart.h"
4 #include "../RTC/bsp_rtc.h"
5 #include "../beep/bsp_beep.h"
6
7 #ifdef USE_LCD_DISPLAY
8 #include "../lcd/bsp_lcd.h"
9 #endif
10
11 /**
12  * @brief 主函数
13  * @param 无
14  * @retval 无
15  */
16 int main(void)
17 {
18     /* 初始化 LED */
19     LED_GPIO_Config();
20     BEEP_GPIO_Config();
21
22     /* 初始化调试串口，一般为串口 1 */
23     Debug_USART_Config();
24     printf("\n\r 这是一个 RTC 闹钟实验 \r\n");
25
26 #ifdef USE_LCD_DISPLAY
27     /*=====液晶初始化开始=====*/
28     LCD_Init();
29     LCD_LayerInit();
30     LTDC_Cmd(ENABLE);
31
32     /*把背景层刷黑色*/
33     LCD_SetLayer(LCD_BACKGROUND_LAYER);
34     LCD_Clear(LCD_COLOR_BLACK);
35
36     /*初始化后默认使用前景层*/
37     LCD_SetLayer(LCD_FOREGROUND_LAYER);
38     /*默认设置不透明，该函数参数为不透明度，范围 0-0xff，0 为全透明，0xff 为不透明*/
39     LCD_SetTransparency(0xFF);
40     LCD_Clear(LCD_COLOR_BLACK);
41     /*经过 LCD_SetLayer(LCD_FOREGROUND_LAYER) 函数后，
42     以下液晶操作都在前景层刷新，除非重新调用过 LCD_SetLayer 函数设置背景层*/
43     LCD_SetColors(LCD_COLOR_RED, LCD_COLOR_BLACK);
44     /*=====液晶初始化结束=====*/
45 #endif
46     /*
47     * 当我们配置过 RTC 时间之后就往备份寄存器 0 写入一个数据做标记
48     * 所以每次程序重新运行的时候就通过检测备份寄存器 0 的值来判断
49     * RTC 是否已经配置过，如果配置过那就继续运行，如果没有配置过
50     * 就初始化 RTC，配置 RTC 的时间。
51     */
52
53     /* RTC 配置：选择时钟源，设置 RTC_CLK 的分频系数 */
```



```
54     RTC_CLK_Config();
55
56     if (RTC_ReadBackupRegister(RTC_BKP_DRX) != 0xf) { //RTC_BKP_DATA
57         /* 闹钟设置 */
58         RTC_AlarmSet();
59
60         /* 设置时间和日期 */
61         RTC_TimeAndDate_Set();
62
63
64     } else {
65         /* 检查是否电源复位 */
66         if (RCC_GetFlagStatus(RCC_FLAG_PORRST) != RESET) {
67             printf("\r\n 发生电源复位....\r\n");
68         }
69         /* 检查是否外部复位 */
70         else if (RCC_GetFlagStatus(RCC_FLAG_PINRST) != RESET) {
71             printf("\r\n 发生外部复位....\r\n");
72         }
73
74         printf("\r\n 不需要重新配置 RTC....\r\n");
75
76         /* 使能 PWR 时钟 */
77         RCC_APB1PeriphClockCmd(RCC_APB1Periph_PWR, ENABLE);
78         /* PWR_CR:DBF 置 1, 使能 RTC、RTC 备份寄存器和备份 SRAM 的访问 */
79         PWR_BackupAccessCmd(ENABLE);
80         /* 等待 RTC APB 寄存器同步 */
81         RTC_WaitForSynchro();
82
83         /* 清除 RTC 中断标志位 */
84         RTC_ClearFlag(RTC_FLAG_ALRAF);
85         /* 清除 EXTI Line 17 悬起位 (内部连接到 RTC Alarm) */
86         EXTI_ClearITPendingBit(EXTI_Line17);
87     }
88
89     /* 显示时间和日期 */
90     RTC_TimeAndDate_Show();
91 }
```

主函数中，我们通过读取备份寄存器的值来判断 RTC 是否初始化过，如果没有则初识话 RTC，并设置闹钟时间，如果已经初始化过，则判断是电源还是外部引脚复位，并清除闹钟相关的中断标志位。

### 3. 下载验证

把编译好的程序下载到开发板，当日历时间到了闹钟时间时，蜂鸣器一直响，但日历会继续运行。